Gradient Boosting & XGBoost

Gradient Boosting

The most important idea to lock in is this:

> **Gradient Boosting does not build Tree 2 to predict y again. Tree 2 is trained to correct the mistakes that the current ensemble is making.**

For regression with squared error, those "mistakes" are literally the residuals:

$$r_i = y_i - \hat{y}_i$$

For other loss functions, they are more generally the **negative gradients of the loss**, so we call them **pseudo-residuals**.

---

1. First understand the big picture

Suppose we have:

```text
X = experience, degree
y = salary
```

Gradient Boosting builds:

```text
Base prediction
      ↓
Tree 1 learns errors
      ↓
Updated prediction
      ↓
Tree 2 learns remaining errors
      ↓
Updated prediction
      ↓
Tree 3 learns remaining errors
      ↓
...
```

Mathematically:

$$F_0(x) = \text{initial/base prediction}$$

Then:

$$F_1(x) = F_0(x) + \eta h_1(x)$$

$$F_2(x) = F_1(x) + \eta h_2(x)$$

and generally:

$$F_M(x) = F_0(x) + \eta \sum_{m=1}^{M} h_m(x)$$

where:

- $F_M(x)$ = final model
- $h_m(x)$ = decision tree m
- η = learning rate

The critical thing is:

> **Each tree is trained using the same input features X, but its target is changed from original y to the current errors/gradients.**

---

2. Why do we need a base model?

Imagine these salaries:

```text
40, 50, 55, 60, 70, 80, 85, 95, 98, 100
```

Before knowing anything about experience or degree, what is the best single salary we can predict?

For squared error:

$$F_0 = \frac{\sum_{i=1} y_i}{n}$$

Here:

$$F_0 = 73.1$$

So initially:

```text
Every person → predicted salary = 73.1
```

Why mean?

Because the constant that minimizes:

$$\sum_i (y_i - c)^2$$

is the mean.

So our starting model is:

$$F_0(x) = 74.4$$

This is the **base model**.

Notice something important:

The base model doesn't have to be a decision tree.

It can simply be a constant.

---

3. Now Tree 1

Here is our dataset.

Row	Experience	Degree	Actual salary
1	1	UG	45
2	2	UG	50
3	3	PG	58
4	4	PG	65
5	5	PG	70
6	6	PhD	82
7	7	PhD	88
8	8	PhD	95
9	9	PG	78

We start with:

$$\hat{y} = 74.4$$

And calculate

Now calculate the residual:

$$r_i = y_i - \hat{y}_i$$

Row	Exp	Degree	Actual y	Base prediction	Residual $y - \hat{y}$
1	1	UG	45	74.1	-29.1
2	2	UG	50	74.1	-24.1
3	3	PG	58	74.1	-16.1
4	4	PG	65	74.1	-9.1
5	5	PG	70	74.1	-4.1
6	6	PhD	82	74.1	7.9
7	7	PhD	88	74.1	13.9
8	8	PhD	95	74.1	20.9
9	9	PG	78	74.1	3.9

Now comes the important part.

---

4. What exactly does Tree 1 learn?

We give the decision tree

```text
X =
    experience
    degree

Target =
    residual
```

NOT:

```text
Target = salary
```

The tree sees:

```text
```

```
experience, degree → residual
```

For example:

```text
Experience = 1, Degree = UG → residual = -26.6
Experience = 2, Degree = UG → residual = -26.6
...
```

The tree tries to find splits that group similar residuals.

For our example, suppose it learns approximately:

```text
                Experience <= 5.5?
                /                \
             Yes                  No
             /                      \
    Experience <= 2.5?          Experience <= 8.5?
      /        \                    /         \
  -26.60     -6.77               11.05       30.60
```

Those leaf values are the **average residuals of the training samples reaching that leaf**.

For example, first leaf:

Rows 1 and 2:

$$\frac{-38.1 + (-31.1)}{2} = -26.6$$

So Tree 1 says:

> "For people in this region of feature space, the current model is approximately **26.6 too high**."

That is what the tree is learning.

---

5. This is where learning rate comes in

Suppose:

$$\eta = 0.1$$

Tree 1 predicts:

$$\cdots$$

$h_1(x)$

But we don't add the entire tree prediction.

We add:

$$0.1h_1(x)$$

So:

$$F_1(x) = F_0(x) + 0.1h_1(x)$$

This prevents one tree from making an excessively large correction.

---

6. Complete Tree 1 calculation

Row	Actual y	Base	Residual	Tree 1 output	0.1 × h_1	New prediction
1	45	74.10	-29.10	-26.60	-13.30	**60.80**
2	50	74.10	-24.10	-26.60	-13.30	**60.80**
3	58	74.10	-16.10	-9.77	-4.88	**69.22**
4	65	74.10	-9.10	-9.77	-4.88	**69.22**
5	70	74.10	-4.10	-9.77	-4.88	**69.22**
6	82	74.10	7.90	11.65	5.83	**79.93**
7	88	74.10	13.90	11.65	5.83	**79.93**
8	95	74.10	20.90	11.65	5.83	**79.93**
9	78	74.10	3.90	11.65	5.83	**79.93**
10	110	74.10	35.90	35.90	17.95	**92.05**

So now we have a much better model.

The prediction changed from:

```text
74.1 for everybody
```

to something based on experience.

---

7. Now Tree 2

Tree 2 trains on these residuals

This is the part people often misunderstand.

We **do not** calculate:

```
Actual salary − Tree 1 output
```

Instead:

> We calculate the error of the **entire current ensemble**.

So:

$$r_i^{(2)} = y_i - F_1(x_i)$$

For example, Row 1:

$$45 - 60.8 = -15.8$$

Row 10:

$$100 - 88.65 = 11.35$$

So Tree 2 receives these new residuals.

Using the actual calculations:

Row	Actual	Prediction after T1	New residual
1	45	60.80	-15.80
2	50	60.80	-10.80
3	58	69.22	-11.22
4	65	69.22	-4.22
5	70	69.22	0.78
6	82	79.93	2.08
7	88	79.93	8.08
8	95	79.93	15.08
9	78	79.93	-1.93

Now Tree 2 learns:

```
X + new residual
```

It might discover that **degree** now helps explain some of the remaining errors.

For our example:

```
                Degree <= 1.33
               /              \
             Yes               No
            /                    \
    Experience <= 1.91      Experience <= 7.91
       /       \               /       \
   -12.61     -1.79          5.08      16.51
```

Again, those numbers are residual corrections.

---

8. Final prediction after Tree 2

Now:

$$F_2(x) = F_1(x) + 0.5 T_2(x)$$

Row	Actual	After T1	T2 output	0.5 × T2	Final prediction
1	45	60.80	-12.61	-6.30	54.50
2	50	60.80	-12.61	-6.30	54.50
3	58	69.22	-12.61	-6.30	62.91
4	65	69.22	-1.79	-0.89	68.32
5	70	69.22	-1.79	-0.89	68.32
6	82	79.93	5.08	2.54	82.46
7	88	79.93	5.08	2.54	82.46
8	95	79.93	16.51	8.26	88.18
9	78	79.93	-1.79	-0.89	79.03

So our final model is:

$$F_2(x) = 71.3 + 0.5 T_1(x) + 0.5 T_2(x)$$

That's gradient boosting in its simplest form.

---

9. Notice what happened to the loss

Using MSE:

$$MSE = \frac{1}{n}\sum_{i}(y_i - \hat{y}_i)^2$$

Initially:

$$MSE = 2356.39$$

After Tree 1:

$$MSE = 1121.53$$

After Tree 2:

$$MSE = 53.68$$

So:

```text
Base model
MSE = 1356.39
        ↓
Tree 1
MSE = 1121.53
        ↓
Tree 2
MSE = 53.68
```

Every new tree is trying to move the model toward lower loss.

---

10. But why are residuals connected to optimization?

This is the deeper concept you should understand for interviews.

Our loss is:

$$L(y, F(x)) = \frac{1}{2}(y - F(x))^2$$

Take derivative with respect to prediction $F(x)$:

$$\frac{\partial L}{\partial F} = F(x) - y$$

The **negative gradient** is:

$$-\frac{\partial L}{\partial F} = y - F(x)$$

And:

$$y - F(x) = \text{residual}$$

Therefore:

> For squared-error regression, residuals are exactly the negative gradients of the loss.

That's why gradient boosting is called **gradient** boosting.

---

11. This gives us the general algorithm

At iteration m:

Step 1 — Current prediction

$$F_{m-1}(x)$$

Step 2 — Calculate loss gradient

$$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}$$

Step 3 — Train a decision tree

Train:

```text
text
```

$X \rightarrow$ pseudo-residuals

to obtain:

$$h_m(x)$$

Step 4 — Add a small correction

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

Step 5 — Repeat.

---

12. Important: Gradient Boosting doesn't normally "update weights" like AdaBoost

This distinction is **very important**.

You may hear:

> "Boosting gives more weight to incorrectly predicted samples."

That's particularly associated with **AdaBoost**.

In gradient boosting:

```text
AdaBoost
→ explicitly updates sample weights

Gradient Boosting
→ calculates gradients/pseudo-residuals
→ fits trees to those gradients
→ updates the model/function
```

So for your interview, don't casually say:

> "Gradient boosting increases the weight of incorrectly predicted rows."

Better:

> "Gradient boosting doesn't necessarily reweight samples explicitly like AdaBoost. It computes the negative gradient of the loss for each observation and trains the next tree to approximate those gradients."

That's a much more accurate explanation.

---

13. What exactly is the decision tree optimizing?

This is another subtle point.

The tree itself is still a **decision tree**.

It is trying to partition the data into leaves where the target values are similar.

But the target is now:

```
residual = y - pseudo_residual
```

For squared-error boosting:

```
Original target:
salary

Tree 1 target:
y = base_prediction

Tree 2 target:
y = current_prediction

Tree 3 target:
y = current_prediction
```

Therefore:

> The tree's job is to discover **regions of feature space where the current model needs a similar correction.**

This is probably the best mental model.

---

14. Think about one new person

Suppose:

```
Experience = 8
Degree = PhD
```

Our model doesn't say:

```
Tree 1 + salary = 95
Tree 2 + salary = 110
```

Instead:

```
Base:
70.3
```

```
Tree 1:
-11.45

Learning rate:
α 0.5

Tree 2:
-16.51

Learning rate:
α 0.5
```

Therefore:

$$16.8 + (0.5)(11.45) + (0.5)(16.51)$$

$$= 33.18$$

The trees are **correction functions**.

---

15. Why multiple small trees instead of one giant tree?

Imagine using one huge decision tree.

It could potentially learn:

```text
Experience + Degree + Salary
```

very aggressively and memorize the training data.

Gradient boosting instead says:

> "I'll make many small corrections."

For example:

```text
Base model
Tree 1
   ↓
+ small correction
   ↓
+ another correction
   ↓
+ another correction
   ↓
+ another correction
```

```
    2
...
```

This makes the model more controllable and allows the ensemble to progressively reduce the loss.

That's where:

- tree depth
- number of trees
- learning rate
- regularization
- subsampling

become important.

---

16. Learning rate vs number of trees

Suppose:

$$\eta = 1$$

A tree's full correction is added.

If:

$$\eta = 0.1$$

only 10% of the correction is added.

For example:

```text
Tree prediction residual = +10
```

With:

```text
learning rate = 1
→ +10

learning rate = 0.5
→ +5

learning rate = 0.1
→ +1
```

Usually:

```text
smaller learning rate
        +
more trees
```

is a common strategy.

The trade-off is:

```text
small η → slower learning, often better controlled
large η → faster learning, potentially overfit more easily
```

---

17. Now the important classification case: Iris

This is where things become slightly different.

Your Iris target is:

```text
0 = setosa
1 = versicolor
2 = virginica
```

This is **multiclass classification**.

You cannot simply do:

$$residual = y - \hat{y}$$

because y isn't a continuous salary.

Instead, gradient boosting can use **log loss / cross-entropy**.

For each observation we have probabilities:

$$P(\text{setosa}), P(\text{versicolor}), P(\text{virginica})$$

and the model produces class scores/logits:

$$F_0(x), F_1(x), F_2(x)$$

Then softmax converts those scores into probabilities.

Then each class probability is obtained by applying the softmax function:

$$p_k = \frac{e^{F_k}}{\sum_j e^{F_j}}$$

---

18. Important difference: multiclass boosting uses multiple trees

This is an important correction to your "2 decision trees" request.

For 3-class Iris, a standard multiclass gradient boosting implementation generally builds:

> **one tree per class per boosting iteration.**

Therefore:

```text
Boosting iteration 1
    Tree 1 → setosa correction
    Tree 2 → versicolor correction
    Tree 3 → virginica correction

Boosting iteration 2
    Tree 4 → setosa correction
    Tree 5 → versicolor correction
    Tree 6 → virginica correction
```

So if you want **2 boosting iterations**, that's typically **6 class-specific trees**, not 2 total trees.

That is the correct way to understand the multiclass case.

---

19. Tiny Iris example

Let's use six Iris observations:

Row	Sepal L	Sepal W	Petal L	Petal W	Actual
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
3	7.0	3.2	4.7	1.4	Versicolor
4	6.4	3.2	4.5	1.5	Versicolor
5	6.3	3.3	6.0	2.5	Virginica

We have three classes:

$$K = 3$$

---

20. Base model for Iris

We initially know nothing.

Because there are equal numbers of each class:

```text
Setosa = 1/3
Versicolor = 1/3
Virginica = 1/3
```

So:

$$P = (1/3, 1/3, 1/3)$$

For every observation:

Row	Setosa p	Versicolor p	Virginica p
1	.333	.333	.333
2	.333	.333	.333
3	.333	.333	.333
4	.333	.333	.333
5	.333	.333	.333

---

21. Now calculate pseudo-residuals

For classification with cross-entropy, conceptually:

$$r_{ik} = y_{ik} - p_{ik}$$

where:

$$y_{ik} = \begin{cases} 1 & \text{if observation belongs to class } k \\ 0 & \text{otherwise} \end{cases}$$

For Row 1:

Actual = Setosa

Therefore:

```
y = [1, 0, 0]
p = [0.333, 0.333, 0.333]
```

So:

$$r = [0.667, -0.333, -0.333]$$

For Row 3:

Actual = Versicolor

$$y = [0, 1, 0]$$

Therefore:

$$r = [-0.333, 0.667, -0.333]$$

And Row 5:

$$r = [-0.333, -0.333, 0.667]$$

Complete table:

Row	Actual	r(Setosa)	r(Versicolor)	r(Virginica)
1	Setosa	**0.667**	-0.333	-0.333
2	Setosa	**0.667**	-0.333	-0.333
3	Versicolor	-0.333	**0.667**	-0.333
4	Versicolor	-0.333	**0.667**	-0.333
5	Virginica	-0.333	-0.333	**0.667**
6	Virginica	-0.333	-0.333	**0.667**

Now we train **three regression trees**.

---

22. Tree for Setosa

Tree 1's target is:

```text
Setosa residual
```

The tree might discover:

```text
Petal length measurements
          ≥
      Setosa?
```

For our tiny dataset, a simple split such as:

```text
Sepal length >= 5.45
```

can separate the two Setosa examples.

The tree therefore predicts approximately:

```text
Setosa tree:

Setosa region      +0.667
Other region       -0.333
```

Similarly:

Versicolor tree

```text
Sepal length >= 6.55
        ≥
Versicolor correction
```

Virginica tree

```text
Petal length < 6.9
        ≥
Virginica correction
```

Again, these are **corrections to class scores**, not final probabilities.

---

23. Apply learning rate

Let's use:

$$\eta = 0.5$$

For Row 1:

```text
Setosa trees       -0.667
Versicolor trees   -0.333
Virginica trees    -0.333
```

Multiply by 0.5:

```text
-0.333
-0.167
-0.167
```

Our initial class scores were:

```text
[0, 0, 0]
```

So new scores become:

$$F = [0.333, -0.167, -0.167]$$

These are **not probabilities yet**.

---

24. Convert scores into probabilities

Use softmax:

$$P_k = \frac{e^{F_k}}{\sum_j e^{F_j}}$$

For Row 1:

```text
```

```
Scores:

Setosa        0.111
Versicolor    0.167
Virginica     0.167
```

Softmax gives approximately:

```text
Setosa        0.452
Versicolor    0.274
Virginica     0.274
```

So prediction:

```text
Setosa
```

because:

$$0.452 > 0.274$$

---

25. After the first boosting iteration

Our six rows become approximately:

Row	Actual	Setosa p	Versicolor p	Virginica p	Prediction
1	Setosa	**0.452**	.274	.274	Setosa
2	Setosa	**0.452**	.274	.274	Setosa
3	Versicolor	.274	**.452**	.274	Versicolor
4	Versicolor	.274	**.452**	.274	Versicolor
5	Virginica	.274	.274	**.452**	Virginica

Notice:

The model has become more confident.

Initially:

```
```

```
15.5 / 15.5 / 15.5
```

After Tree round 1:

```text
≈0.2 / 27.4 / 27.4
```

---

26. Then the second boosting iteration

Now we **do not go back to the original residuals**.

We calculate new probabilities:

```text
P = [0.052, 0.374, 0.374]
```

Then calculate new pseudo-residuals:

$$r = y - p$$

For a Setosa row:

$$r = [1 - .052,\ 0 - .374,\ 0 - .374]$$

$$r = [0.948, -0.374, -0.374]$$

For a Versicolor row:

$$r = [-0.374, 0.626, -0.374]$$

For a Virginica row:

$$r = [-0.374, -0.374, 0.626]$$

So the new trees now have smaller corrections because the model is already moving in the right direction.

---

27. Second round

The three trees produce approximately:

```text
```

```
Setosa trees:
    Setosa region       -0.516
    other regions        0.171

Versicolor trees:
    Versicolor region   -0.108
    other regions        0.171

Virginica trees:
    Virginica region    -0.108
    other regions        0.171
```

After learning rate:

$$0.5 \times 0.109 = 0.271$$

So for a Setosa example, the score becomes:

$$F_{\text{Setosa}} = 0.333 + 0.271 = 0.603$$

and the other two:

$$F = -0.167 - 0.121 = -0.301$$

So:

```
Scores:

Setosa      0.687
Versicolor  0.184
Virginica   0.184
```

Softmax gives approximately:

```
Setosa      0.551
Versicolor  0.221
Virginica   0.221
```

Again:



---

28. Final Iris example

After two boosting iterations:

Row	Actual	Setosa	Versicolor	Virginica	Final class
1	Setosa	**0.554**	.223	.223	**Setosa**
2	Setosa	**0.554**	.223	.223	**Setosa**
3	Versicolor	.223	**.554**	.223	**Versicolor**
4	Versicolor	.223	**.554**	.223	**Versicolor**
5	Virginica	.223	.223	**.554**	**Virginica**

The classification decision is simply:



So if:

```text
Setosa      = 0.223
Versicolor  = 0.554
Virginica   = 0.223
```

then:

```text
prediction = Versicolor
```

29. Salary vs Iris — put them side by side

This is the connection I want you to remember.

Concept	Salary regression	Iris classification
Target	Continuous salary	Class
Loss	MSE	Cross-entropy/log loss
Base model	Mean salary	Initial class scores/prior probabilities
Tree target	Residual	Pseudo-residual / negative gradient

Tree type	Regression tree	Regression trees on gradients
Output	Correction to salary	Correction to class score
Learning rate	Shrinks correction	Shrinks correction
Final output	Numeric salary	Class probabilities

30. The most important mental model

Don't think:

```text
Tree 1 predicts salary
Tree 2 predicts salary
Tree 3 predicts salary
```

Instead think:

```text
Raw model
    |
"What's wrong?"
    |
Tree 1 learns correction
    |
Updated model
    |
"What's still wrong?"
    |
Tree 2 learns correction
    |
Updated model
    |
"What's still wrong?"
    |
Tree 3 learns correction
    |
...
```

For regression:

```text
Residual = actual - current prediction
```

For general gradient boosting:

```text
```

```
Pseudo residual = negative gradient of loss
```

For classification:

```text
current probabilities
        ↓
negative gradient
        ↓
class-specific trees
        ↓
class scores
        ↓
softmax
        ↓
probabilities
        ↓
argmax
        ↓
predicted class
```

---

31. One subtle but VERY important point

Suppose the original salary is:

```text
y = 110
```

and after Tree 1:

```text
prediction = 92.85
```

The residual is:

$$110 - 92.85 = 17.85$$

Tree 2 doesn't know:

> "This person's salary is 110."

It is effectively being told:

> "For this person's feature region, the current model needs approximately **+17.85** more prediction."

> residual predictions.

And the tree learns patterns across **all rows** to figure out:

> "Which combinations of experience/degree tend to need positive or negative corrections?"

That is why the **same input features X** are used again, but the **target given to the tree changes at every iteration.**

---

32. What should you say in an interview?

A strong experienced-level answer would be:

> **"Gradient boosting starts with a base model, typically the mean for squared-error regression. At every iteration, we calculate the negative gradient of the loss with respect to the current predictions; for MSE this is simply the residual $y - \hat{y}$. We then train a shallow decision tree on the original features but use these pseudo-residuals as the target, so the tree learns where and how the current ensemble needs correction. Its output is scaled by the learning rate and added to the existing model, and we repeat this process until the desired number of trees or stopping criteria is reached."**

And for classification:

> **"For multiclass classification, the model maintains class scores, converts them to probabilities using softmax, computes class-wise negative gradients from cross-entropy, and typically fits one regression tree per class at each boosting iteration. The trees update the class scores, which are converted back to probabilities, and the class with the highest final probability becomes the prediction."**

The 5 formulas you should memorize

1. Base regression model

$$F_0 = \arg\min_{\gamma} \sum_{i=1}^{n} L(y_i, \gamma)$$

For MSE:

$$F_0 = \text{mean}(y)$$

2. Residual / negative gradient

$$r_{im} = -\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}$$

For MSE:

$$r = y - F(x)$$

3. Tree fitting

$$h_m(x) \approx r_m(x)$$

4. Model update

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

5. Multiclass prediction

$$P_k = \frac{e^{F_k}}{\sum_j e^{F_j}}$$

and:

$$\hat{y} = \arg\max_k P_k$$

If you understand those five equations and the salary table above, you understand the core mechanics of Gradient Boosting rather than just memorizing the definition.

XGBoost

Yes. The easiest way to understand **XGBoost** is:

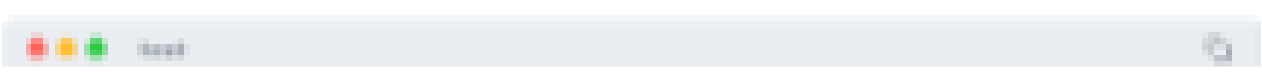
> XGBoost is still gradient boosting, but it makes the tree-building process much more sophisticated and adds explicit regularization, second-order optimization, efficient split finding, parallelism, and handling of sparse/missing data.

I'll use the same salary example first, then connect each difference to the actual XGBoost mathematics.

---

1. First: Gradient Boosting vs XGBoost

Think of standard Gradient Boosting as:



```
Base prediction
      ↓
Calculate residual / negative gradient
      ↓
Build decision tree on residuals
      ↓
Multiply by learning rate
      ↓
Add tree to ensemble
      ↓
Repeat
```

XGBoost keeps this basic idea:

```text
Current prediction
      ↓
Calculate gradient + Hessian
      ↓
Find the best tree structure
      ↓
Calculate optimal leaf weights
      ↓
Regularize the tree
      ↓
Add tree × learning rate
      ↓
Repeat
```

The major difference is:

Gradient Boosting

Mostly asks:

> "What residual should my next tree learn?"

XGBoost

Asks:

> "Given the current predictions, gradients, and curvature of the loss, what tree structure and leaf values will produce the greatest regularised reduction in objective?"

That's a much more optimisation-oriented approach.

---

2. Same salary example

Our data:

Row	Experience	Degree	Salary
1	1	UG	45
2	2	UG	50
3	3	PG	58
4	4	PG	65
5	5	PG	70
6	6	PhD	82
7	7	PhD	88
8	8	PhD	95
9	9	PG	78
10	10	PhD	110

Initial prediction:

$$\hat{y}_0 = 74.1$$

Exactly as before.

---

3. Gradient Boosting would calculate residuals

For squared error:

$$r_i = y_i - \hat{y}_i$$

So:

Row	y	Prediction	Residual
1	45	74.1	-29.1
2	50	74.1	-24.1
3	58	74.1	-16.1
4	65	74.1	-9.1
5	70	74.1	-4.1
6	82	74.1	7.9
7	88	74.1	13.9



8	95	74.1	20.9
9	70	74.1	-3.9
10	110	74.1	35.9

Standard gradient boosting might train a tree directly against these residuals.

XGBoost says:

> "Let's use more information than just the residual."

---

4. XGBoost uses first AND second derivatives

This is one of the most important differences.

For each observation, XGBoost calculates:

First derivative

$$g_i = \frac{\partial L(y_i, \hat{y}_i)}{\partial \hat{y}_i}$$

Second derivative

$$h_i = \frac{\partial^2 L(y_i, \hat{y}_i)}{\partial \hat{y}_i^2}$$

The first derivative tells us:

> **Which direction should I move?**

The second derivative tells us:

> **How does the loss curve behave around the current prediction?**

You can think of:

```text
Gradient
   ↓
direction of correction

Hessian
   ↓
curvature / how aggressively the correction should be made
```

---

5. For squared error

Let's use:

$$L = \frac{1}{2}(y - \hat{y})^2$$

Derivative:

$$g_i = \hat{y}_i - y_i$$

Second derivative:

$$h_i = 1$$

So for Row 1:

$$g_1 = 71.1 - 65 = 38.1$$

$$h_1 = 1$$

Row 10:

$$g_{10} = 15.1 - 110 = -95.0$$

$$h_{10} = 1$$

Notice something interesting.

Our previous residual was:

$$r = y - \hat{y}$$

while XGBoost's gradient is:

$$g = \hat{y} - y$$

Therefore:

$$\boxed{g = -r}$$

That's because XGBoost uses the gradient direction, whereas gradient boosting often describes the target as the **negative gradient**.

---

6. Our complete XGBoost first-order/second-order table

Row	y	Current prediction	Gradient g_i	Hessian h_i

1	45	74.1	29.1	1
2	50	74.1	24.1	1
3	58	74.1	16.1	1
4	65	74.1	9.1	1
5	70	74.1	4.1	1
6	82	74.1	-7.9	1
7	88	74.1	-13.9	1
8	95	74.1	-20.9	1
9	70	74.1	-3.9	1

For squared error, Hessian is always 1, so it doesn't look very exciting.

But for losses such as **log loss**, Hessian varies by observation and becomes much more useful.

---

7. Now comes the biggest XGBoost difference: leaf weight calculation

Suppose XGBoost considers putting some observations into a leaf.

For a leaf j, calculate:

$$G_j = \sum_{i \in j} g_i$$

and:

$$H_j = \sum_{i \in j} h_i$$

Then the optimal leaf weight is:

$$\boxed{w_j^* = -\frac{G_j}{H_j + \lambda}}$$

where:

$$\lambda$$

is L2 regularisation.

This is a major difference from the simple residual tree explanation.

---

8. Example: first leaf

Suppose a candidate split creates:

```
leaf A = rows 1,3
```

Their gradients:

$$59.1,\ 31.1$$

Therefore:

$$G_A = 59.1 + 31.1 = 43.2$$

Hessians:

$$H_A = 1 + 1 = 2$$

Suppose:

$$\lambda = 10$$

Then:

$$w_A = -\frac{43.2}{2 + 10}$$

$$w_A = -3.133$$

So XGBoost's leaf prediction is approximately:

$$\boxed{-3.13}$$

Notice what happened.

A standard regression tree on residuals might have predicted:

$$\text{mean residual} = \frac{-29.1 - 31.1}{2} = -30.6$$

XGBoost isn't simply saying:

> "Leaf prediction = average residual."

It calculates an **optimised regularised leaf weight** using gradients, Hessians and λ.

9. Why is the sign different?

Remember:

$$g = \hat{y} - y$$

For Rows 1 and 2:

$$G = 53.3$$

Therefore:

$$w = -1.13$$

That's a negative correction, which makes sense because our current prediction of 74.1 is too high for those people.

So:

```text
Current prediction
74.1

XGBoost correction
-1.13
```

With learning rate:

$$\eta = 0.1$$

actual update:

$$0.1(-1.13) = -0.113$$

giving:

$$73.613$$

This is intentionally a very small correction because we chose a strong regularization value and small learning rate.

10. Regularization — one of the biggest differences

XGBoost explicitly regularizes the tree.

Its objective can be thought of as:

$$\text{Objective} = \text{Loss} + \text{TreeComplexity}$$

More formally:

$$\mathcal{O}(y) = \sum_{i}^{m} L(y_i, \hat{y}_i) + \sum_{k}^{n} \Omega(f_k)$$

where:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j}^{T} w_j^2 + \alpha \sum_{j}^{T} |w_j|$$

Conceptually:

```text
Training loss
    -
penalty for complicated trees
    -
penalty for large leaf values
```

---

11. What do γ, λ and α do?

γ — tree/split complexity

$$\gamma T$$

A new split must provide enough improvement to justify its complexity.

So if a split barely improves the objective:

```text
Gain < γ
```

XGBoost can reject the split.

λ — L2 regularisation

$$\frac{1}{2}\lambda \sum_{j}^{m} w_j^2$$

It discourages very large leaf weights.

Increasing λ:

```
text
```

```
large correction
      ↓
smaller correction
```

α — L1 regularization

$$\alpha \sum_j |w_j|$$

This can push some leaf weights toward zero and is useful for additional sparsity/regularization.

---

12. How does XGBoost decide whether a split is good?

This is another major difference.

Suppose we have:

```
text
```

```
Parent node
     |
     | split
     ↓
  Left      Right
```

XGBoost calculates:

$$G_L, H_L$$

and:

$$G_R, H_R$$

The approximate split gain is:

$$\boxed{\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}\right] - \gamma}$$

where:

$$G = G_L + G_R$$

and:

$$H = H_L + H_R$$

This formula is **central to understanding XGBoost**.

---

13. What is XGBoost actually doing while building the tree?

Suppose it considers:

```text
Experience <= 1.5
Experience <= 2.5
Experience <= 5.5
Experience <= 6.5
...
```

For each candidate split it calculates:

```text
left:
    sum gradients
    sum Hessians

right:
    sum gradients
    sum Hessians

        ↓

calculate split gain

        ↓

choose split with highest positive gain
```

So XGBoost isn't just blindly growing a CART tree.

It is using the **gradient statistics of the current boosting iteration to select splits**.

---

14. Gradient Boosting vs XGBoost tree construction

This is a useful interview comparison.

Traditional Gradient Boosting

Conceptually:

```text
Calculate residuals
      ↓
Use residuals to decision tree algorithm
      ↓
Tree finds splits that reduce residual squared error
      ↓
Tree predicts residual
```

XGBoost

```text
Calculate gradients + Hessians
          ↓
For every candidate split
          ↓
Calculate G and H
          ↓
Calculate regularized gain
          ↓
Choose best split
          ↓
Calculate optimal regularized leaf weights
```

So XGBoost incorporates the optimization objective directly into tree construction.

---

15. Parallel processing

Another major XGBoost advantage is computational efficiency.

A naive tree-building process might look like:

```text
Feature 1
   ↓
try splits

Feature 2
   ↓
try splits

Feature 3
   ↓
try splits
```

XGBoost can perform much of the split evaluation in parallel.

For example:

```text
                Training data
                     |
      ┌──────────────┼──────────────┐
      |              |              |
  Feature 1      Feature 2      Feature 3
      |              |              |
  Histogram      Histogram      Histogram
      |              |              |
    gains          gains          gains
      └──────────────┼──────────────┘
                     |
                 best split
```

Modern XGBoost implementations use highly optimized tree-building algorithms, including histogram-based methods, to reduce the amount of computation and memory required.

---

16. Why can split finding be parallelized?

Suppose we have:

```text
1 million rows
100 features
```

For each feature, we need information such as:

$$\sum_{i \in R} g_i$$

and:

$$\sum_{i \in R} h_i$$

for candidate split regions.

These calculations can be performed efficiently using parallel computation.

So rather than:

```text
```

```
feature 1 → Redis
feature 2 → Redis
feature 3 → Redis
...
```

we can process feature/split statistics in parallel.

This doesn't mean:

> "All trees are trained simultaneously."

That's an important distinction.

Trees within the boosting sequence are dependent.

Tree 2 needs predictions from Tree 1.

Therefore:

```text
Tree 1
  ↓
Tree 2
  ↓
Tree 3
```

cannot simply all be trained independently.

The parallelism is primarily **within the construction of each tree**, especially feature/split statistics and histogram operations.

---

17. Sparsity-aware splitting

This is particularly important in real-world enterprise data.

Imagine:

```
Customer data

age
income
4G_usage
internet_consult_reading
...
```

You could have:

```
text
```

```
fill_usage = 0
international_roaming = missing
special_offer = missing
```

Large ML datasets can have many:

- missing values
- zero values
- one-hot encoded features
- sparse matrices

A naive algorithm may waste huge amounts of computation processing all those zeros/missing values.

XGBoost is designed to exploit sparsity.

---

18. What does sparsity-aware mean?

Suppose:

```
text
```

```
feature = international_roaming
```

Dataset:

Customer	Feature
1	1
2	0
3	0
4	missing
5	0
6	1
7	missing
8	0

Most values are zero/missing.

Instead of treating every zero as an ordinary numerical value and doing unnecessary

work, xgboost can efficiently handle the sparse representation.

For missing values, XGBoost learns a **default direction** for the missing values when evaluating a split.

Conceptually:

```text
Feature == missing?

          Split
            |
      ┌─────┴─────┐
      ↓           ↓
    left        right

Missing?
   ↓
Learn whether
left or right
gives better gain
```

So the algorithm can learn:

```text
missing → left
```

or:

```text
missing → right
```

depending on which gives better objective improvement.

---

19. Why is this valuable?

Imagine a telecom dataset with:

```text
100 million customers
1,000 features
```

where many features represent things such as:

```text
```

```
roaming_usage
international_calls
OTT_subscription
device_type
special_service
```

Many customers may have:

```text
0 usage
```

or missing values.

Efficient sparse handling can significantly reduce the computational burden.

This is one reason XGBoost became extremely popular for large tabular datasets.

---

20. Put everything together

Now compare the two algorithms using our salary example.

Gradient Boosting

```
Base prediction
↓
Residuals
y − prediction
↓
Train regression tree
↓
Tree predicts residual
↓
× learning rate
↓
Add correction
↓
Repeat
```

XGBoost

```
Base prediction
↓
```

```
      ↓
Calculate:
    gradient g
    Hessian h
      ↓
For candidate splits:
    calculate G and H
      ↓
Calculate regularized gain
      ↓
Choose best split
      ↓
Calculate optimal leaf weight
      ↓
Apply regularization
      ↓
× learning rate
      ↓
Update prediction
      ↓
Repeat
```

21. The same example in one table

Aspect	Gradient Boosting	XGBoost
Basic idea	Sequential error correction	Sequential error correction
Tree target	Residual / negative gradient	Gradient + Hessian used in optimization
First derivative	Used conceptually	**Explicitly used**
Second derivative	Generally not used	**Explicitly used**
Leaf value	Often mean residual	**Optimal regularized leaf weight**
Regularization	More limited/basic depending on implementation	**Explicit L1/L2 + tree complexity regularization**
Split selection	Tree impurity/error reduction	**Gradient/Hessian-based regularized gain**
Learning rate	Yes	Yes
Number of trees	Yes	Yes
Regularization	More limited in classic	**Highly optimized/parallel**

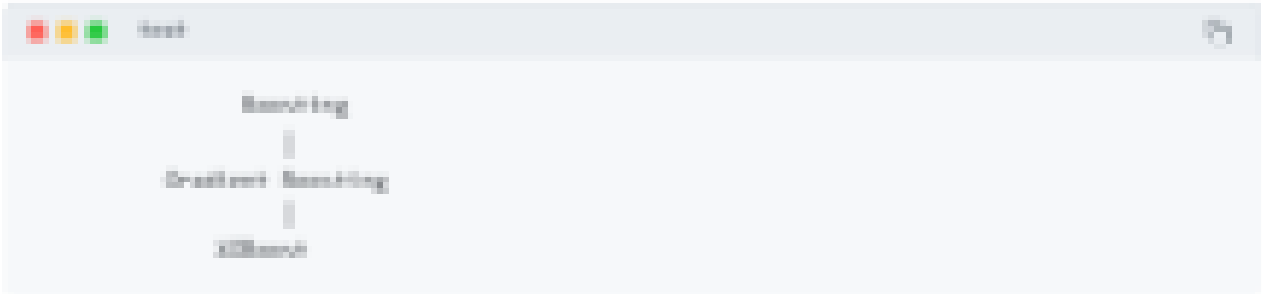
	Implementations	**tree construction**
Sparse data	Basic handling depends on implementation	**Sparsity-aware algorithms**
Missing values	Depends on implementation	**Learns default split direction**
Computation	Can be slower	Highly optimized
Main	Fit residuals	Optimize regularized objective

22. One subtle point: XGBoost is still gradient boosting

Don't explain them as completely different algorithms.

That's incorrect.

The relationship is:



More accurately, XGBoost is an **optimized, regularized gradient boosting framework** that uses second-order information and efficient systems techniques.

So if an interviewer asks:

> "What's the difference between Gradient Boosting and XGBoost?"

Don't say:

> "Gradient boosting uses residuals while XGBoost uses gradients."

That's misleading because gradient boosting itself is based on gradients.

Instead say:

> **"XGBoost follows the same additive gradient boosting framework, but differs in how aggressively it optimizes the objective. It uses both first- and second-order derivatives, explicitly regularizes tree complexity and leaf weights, uses gradient/Hessian-based split gain, and introduces highly optimized parallel and sparsity-aware tree construction."**

23. And this connects directly to your previous

confusion

You asked earlier:

> "What exactly is the tree trying to achieve?"

For **Gradient Boosting**:



For **XGBoost**:



That's the key conceptual upgrade.

The progression is:

```text
Gradient Boosting:

"What errors should I correct?"
          ↓
    residuals/gradients
          ↓
        tree


XGBoost:

"What correction and tree structure
will reduce my loss the most,
while controlling model complexity?"
          ↓
    gradients + Hessians
          ↓
    split gain + regularization
          ↓
    optimal leaf weights
          ↓
        tree
```

If you're preparing for an experienced AI/ML interview, remember these four XGBoost keywords:

1. First derivative → direction

2. Second derivative → curvature

3. Regularized gain → whether a split is worth taking

4. Sparsity-aware → efficiently handle sparse/missing features

And one final distinction:

> **Parallelization speeds up construction of an individual tree; it does not remove the sequential dependency between boosting rounds.**